

WEB APPLICATION PENETRATION TEST REPORT

SQL Injection — UNION-Based Database Fingerprinting

Target Application

PortSwigger Web Security Academy — "We Like To Shop" Lab Environment

Prepared by	Mohammed Ashraf Mokhtar
LinkedIn	linkedin.com/in/1mohammed-ashraf
GitHub	github.com/moashraf2006
Engagement type	Authorized security training lab (PortSwigger Web Security Academy)
Assessment date	2 September 2026
Report classification	Confidential

This report documents a controlled penetration test performed against a PortSwigger Web Security Academy training lab, used strictly for educational purposes.

Table of Contents

1. Executive Summary
2. Engagement Scope
3. Methodology
4. Tools Utilized
5. Finding: SQL Injection — UNION-Based Database Fingerprinting
 - 5.1 Summary
 - 5.2 Severity
 - 5.3 Affected Component
 - 5.4 Steps to Reproduce
 - 5.5 Impact
 - 5.6 Remediation
6. Conclusion

1. Executive Summary

This report presents the findings of a penetration test conducted against a deliberately vulnerable e-commerce web application ("We Like To Shop"), hosted as a PortSwigger Web Security Academy practitioner-level lab. The objective of the engagement was to identify and exploit a SQL Injection vulnerability within the product category filtering functionality, and to demonstrate the ability to fingerprint the underlying database engine and version through a UNION-based attack.

During testing, a critical SQL Injection vulnerability was identified in the category parameter of the /filter endpoint. The application concatenates user-supplied input directly into a backend SQL query without sanitization or parameterization. This flaw was successfully exploited using a UNION SELECT attack to determine that the query returns two columns and to extract the database version string directly into the application's rendered response, without requiring any authentication.

The vulnerability was successfully exploited and validated as fixed by the lab platform, which returned a "Congratulations, you solved the lab!" confirmation. The finding is rated Critical due to the ease of exploitation, the lack of authentication required, and the potential for full database compromise in a real-world deployment of this pattern.

Key Findings

- One (1) Critical-severity SQL Injection vulnerability identified in the category filter parameter.
- The vulnerability allows an unauthenticated attacker to execute arbitrary UNION-based SQL queries.
- The database engine was successfully fingerprinted as MySQL, version 8.0.42-0ubuntu0.20.04.1.
- No input validation, parameterized queries, or WAF protections were observed on the affected endpoint.

Recommendations Summary

- Refactor all database access to use parameterized queries / prepared statements.
- Apply strict server-side allow-list validation to the category parameter.
- Deploy a Web Application Firewall (WAF) as a defense-in-depth control.
- Suppress verbose backend error messages (e.g. "Internal Server Error" stack traces) returned to the client.

2. Engagement Scope

Target application	"We Like To Shop" demo storefront
Target host	0a6e00fc0484cb4981d920560093002b.web-security-academy.net
In-scope functionality	Product category filter (/filter?category=)
Testing type	Black-box, unauthenticated, manual dynamic testing

Testing window	2 September 2026
Authorization	Testing performed on a PortSwigger-provisioned lab instance intended for authorized security training

3. Methodology

Testing followed a standard black-box web application methodology consistent with industry-recognized approaches such as the OWASP Testing Guide, adapted to the scope of a single, targeted lab objective:

- **Reconnaissance** — Reviewed the lab brief and objective to understand the required exploitation outcome.
- **Enumeration** — Browsed the target application and identified the category filtering parameter as a user-controlled input reflected in a backend query.
- **Vulnerability Discovery** — Intercepted filter requests using Burp Suite and manipulated the category parameter to test for SQL injection.
- **Exploitation** — Iteratively refined a UNION-based SQL injection payload to determine column count, valid comment syntax, and to extract data via the @@version system variable.
- **Validation** — Confirmed successful exploitation via both Burp Repeater and the live browser session, and validated the lab's "Solved" status.
- **Reporting** — Documented reproduction steps, evidence, impact, and remediation guidance in this report.

4. Tools Utilized

Interception proxy	Burp Suite Community Edition v2026.7.3
Browser	Microsoft Edge (Chromium-based)
Target platform	PortSwigger Web Security Academy

5. Finding: SQL Injection — UNION-Based Database Fingerprinting

5.1 Summary

The category parameter of the product filter endpoint (GET /filter?category=) is vulnerable to SQL Injection. User input is inserted directly into a SQL query executed against the backend MySQL database without sanitization. By using a UNION SELECT attack, it was possible to determine the number of columns returned by the original query and subsequently retrieve the database version using the @@version system variable, without any authentication.

5.2 Severity

CRITICAL

CVSS v3.1 Vector (illustrative): AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N — **Base Score: 9.1 (Critical)**

The vulnerability is rated Critical because it is remotely exploitable over the network, requires no authentication or user interaction, is trivial to execute (low attack complexity), and permits an attacker to read arbitrary data from the underlying database via UNION-based extraction.

5.3 Affected Component

Endpoint	GET /filter?category={INJECTABLE}
Parameter	category
Injection type	SQL Injection — UNION attack, error-based confirmation
Database engine	MySQL 8.0.42-0ubuntu0.20.04.1
Authentication required	None

5.4 Steps to Reproduce

Step 1 — Review the lab objective

The engagement began by reviewing the PortSwigger lab brief, which confirms that the category filter contains a SQL injection vulnerability exploitable via a UNION attack, with the goal of retrieving the database version string.

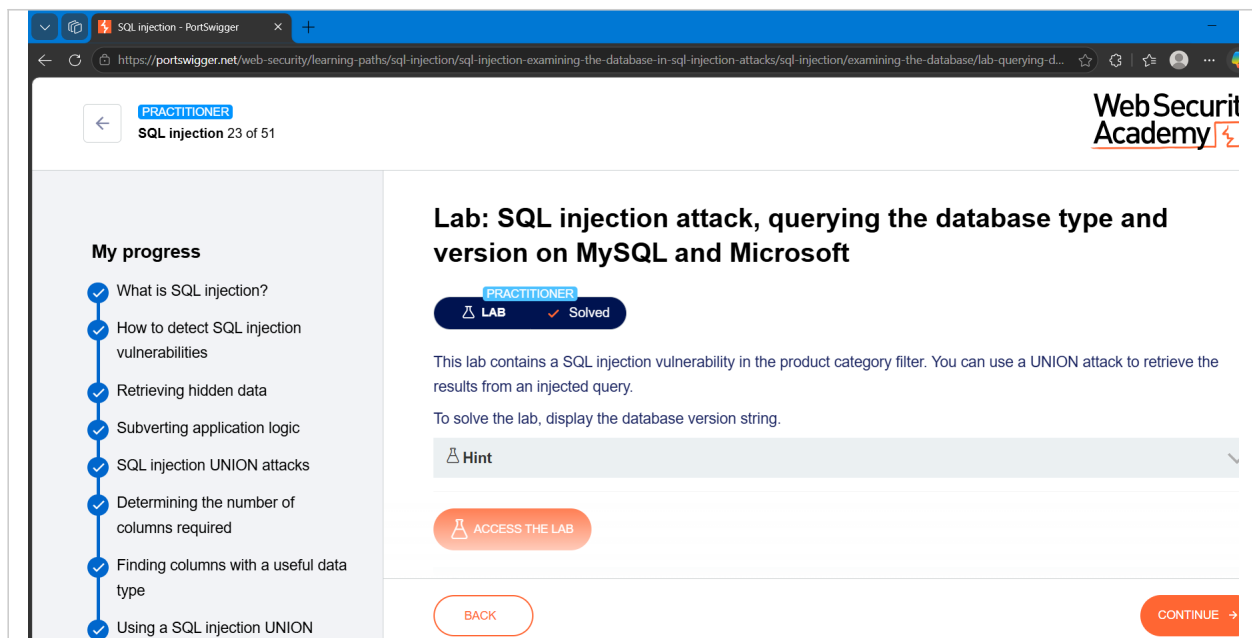


Figure 1. PortSwigger lab brief for "SQL injection attack, querying the database type and version on MySQL and Microsoft", confirming the objective and lab access.

Step 2 — Browse the target application

The lab was accessed and the storefront home page ("We Like To Shop") was reviewed to identify user-controlled parameters, including the category filtering links exposed in the "Refine your search" navigation.

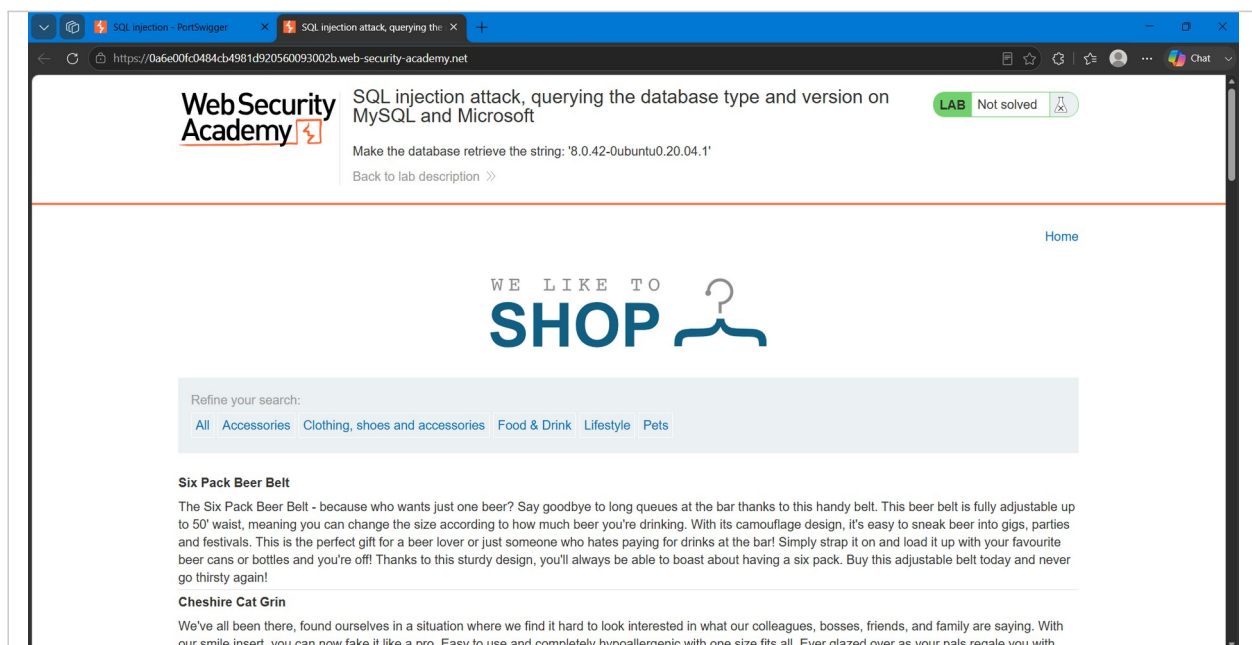


Figure 2. Target storefront home page, showing the category filter links used to reach the vulnerable endpoint.

Step 3 — Identify the injectable parameter

Selecting the "Pets" category revealed the underlying request structure, GET /filter?category=Pets, confirming that the category value is passed directly as a URL parameter and is a strong candidate for a server-side SQL query.

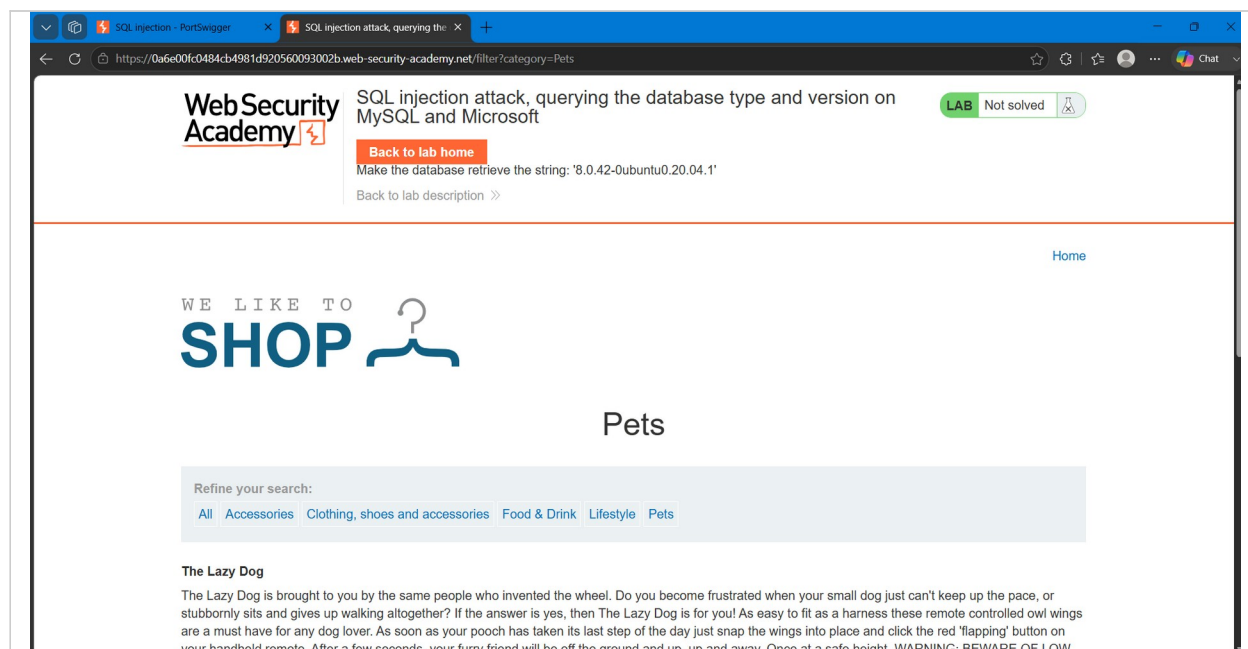


Figure 3. Category filter applied via the URL parameter category=Pets, exposing the injectable input point.

Step 4 — Intercept the request in Burp Suite

Burp Suite's intercepting proxy was used to capture the outgoing GET /filter?category=pets request, confirming the exact parameter name and value that would be manipulated for injection testing.

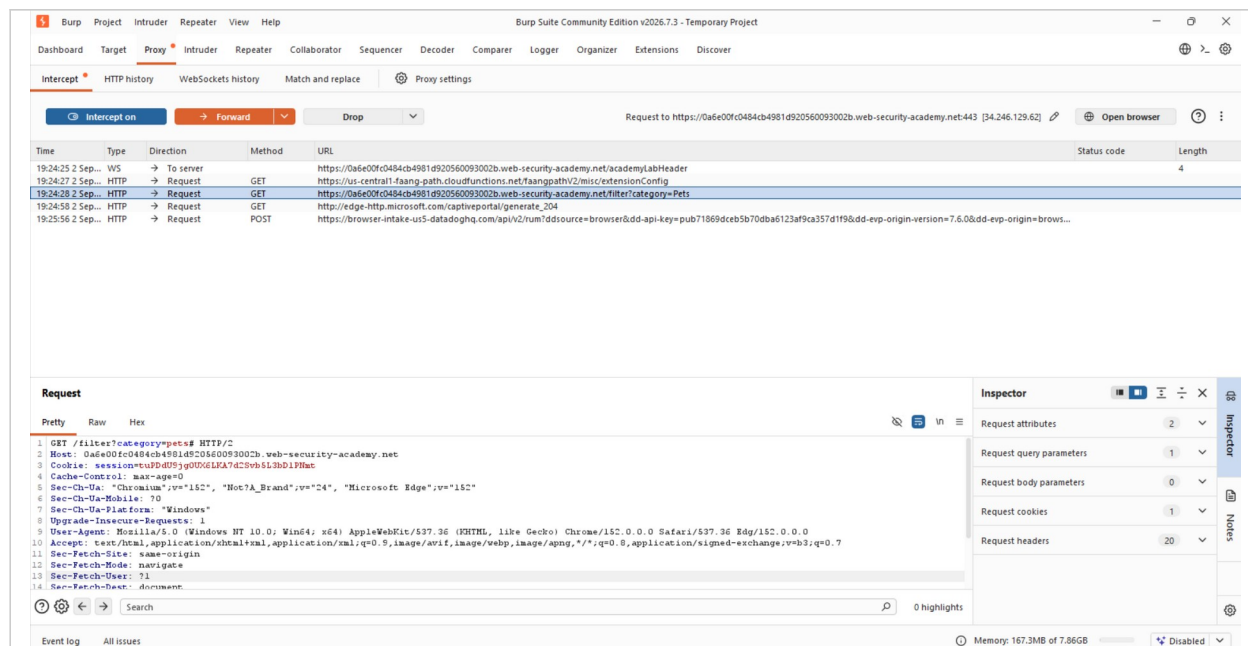


Figure 4. Burp Suite Proxy intercepting the GET /filter?category=pets request prior to injection testing.

Step 5 — Send the request to Repeater

The intercepted request was forwarded to Burp Repeater, where the category parameter value could be modified and resent repeatedly to iteratively test injection payloads and observe the application's response.

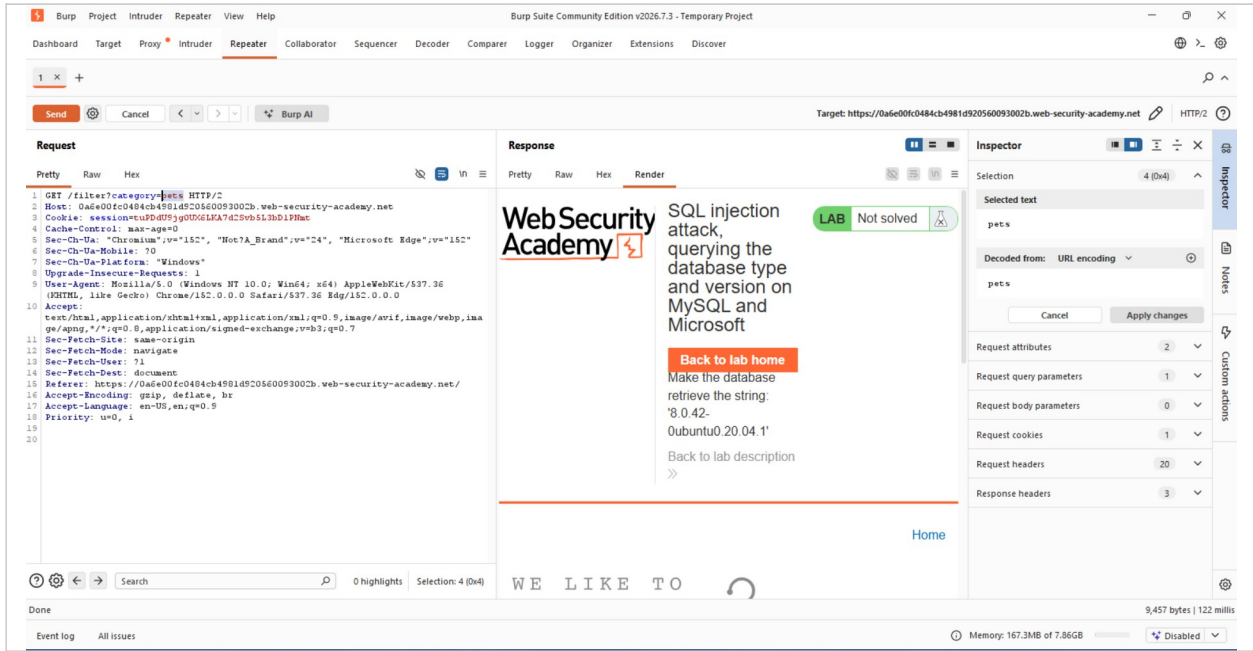


Figure 5. Request sent to Burp Repeater with the category parameter isolated for payload testing via the Inspector panel.

Step 6 — Determine the required comment syntax

An initial payload of ' UNION SELECT NULL,NULL-- was submitted to close the original query and append a UNION SELECT clause. This payload caused an Internal Server Error, indicating that the double-dash comment syntax was not being interpreted correctly by the backend (most likely due to a missing trailing space being stripped, or the backend expecting an alternative comment delimiter).

```
' UNION SELECT NULL,NULL--
```

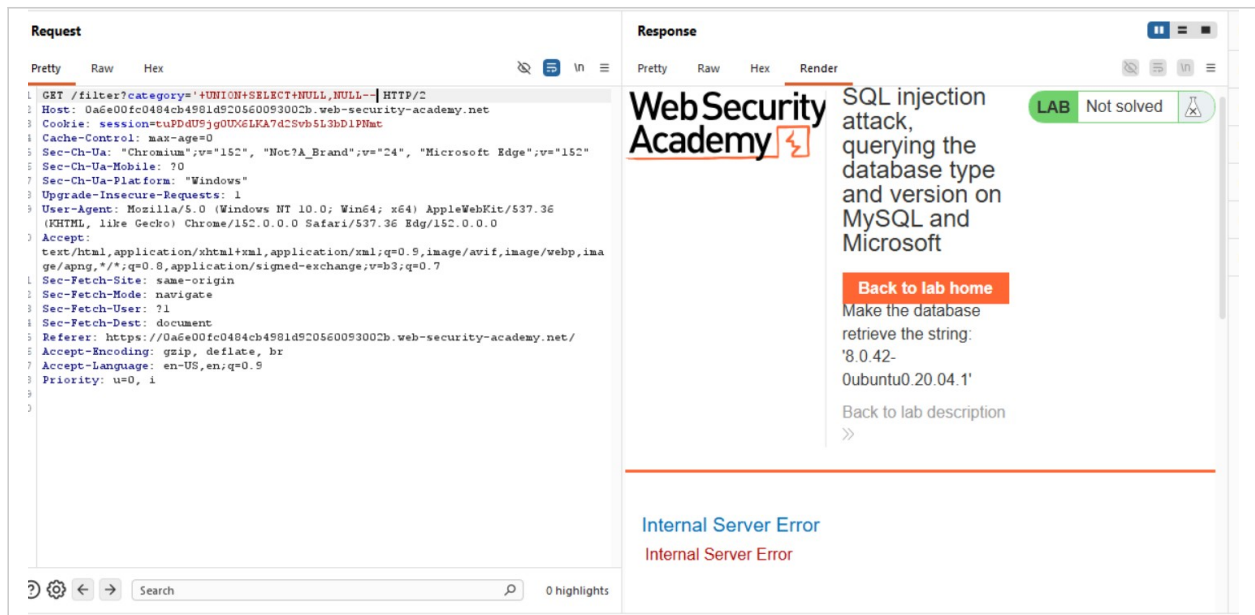



Figure 6. Payload using '--' comment syntax returns an Internal Server Error, indicating the syntax was rejected by the backend query.

Step 7 — Confirm the column count

Replacing the trailing comment with the MySQL hash-style comment (#) resolved the syntax error. The payload ' UNION SELECT NULL,NULL# was accepted by the application and rendered successfully, confirming that the original query returns exactly two columns and that # is the correct comment delimiter for this backend.

```
' UNION SELECT NULL,NULL#
```

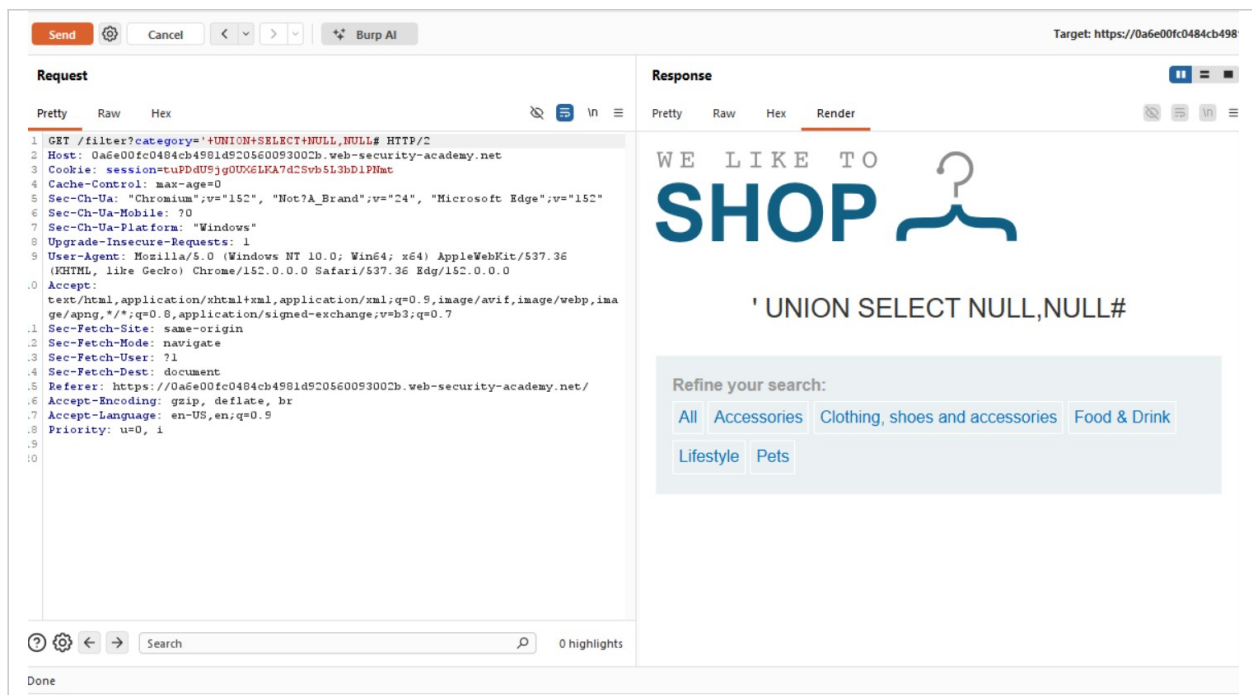


Figure 7. Payload ' UNION SELECT NULL,NULL# executes successfully, confirming a two-column result set.

Step 8 — Extract the database version

With the column count confirmed, the second column was replaced with the MySQL system variable @@version to extract the database version string directly into the rendered page output.

```
' UNION SELECT @@version,NULL#
```

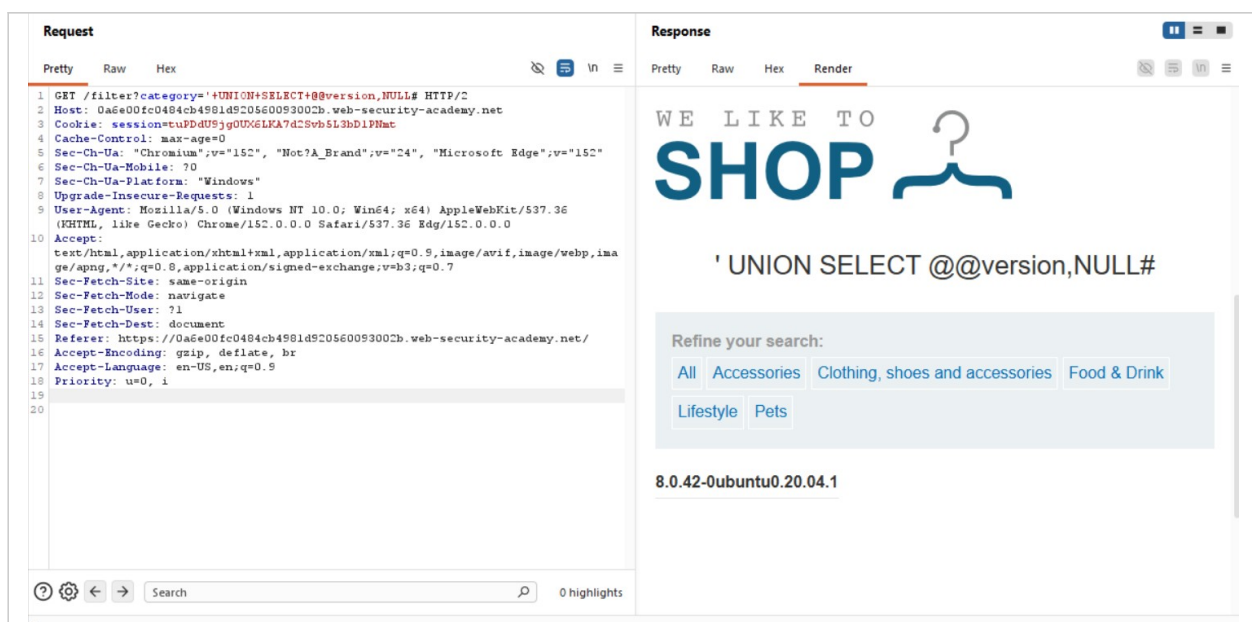


Figure 8. Repeater request using ' UNION SELECT @@version,NULL# — the rendered response discloses the database version, 8.0.42-0ubuntu0.20.04.1.

Step 9 — Validate exploitation in the browser and confirm the lab is solved

The confirmed payload was submitted through the live browser session via the category URL parameter. The application rendered the extracted database version string (8.0.42-0ubuntu0.20.04.1) on the page, and the PortSwigger lab banner updated to "Congratulations, you solved the lab!", formally validating successful exploitation.



Figure 9. Final exploitation confirmed in-browser: the database version is disclosed on the page and the lab status changes to solved.

5.5 Impact

This vulnerability allows an unauthenticated remote attacker to inject and execute arbitrary SQL via the UNION operator. While this proof of concept was limited to database fingerprinting (extracting the engine and version via @@version), the same injection point could realistically be extended to:

- Enumerate database schema, table, and column names via information_schema queries.
- Extract sensitive data such as user credentials, session tokens, or payment information.
- In misconfigured environments, escalate to write access, file read/write (e.g. MySQL LOAD_FILE/INTO OUTFILE), or full database compromise.
- Aid further attacks by disclosing precise backend software versions, enabling targeted exploitation of known CVEs for that version.

5.6 Remediation

- Use parameterized queries / prepared statements for all database access; never concatenate user input directly into SQL statements.
- Apply strict server-side allow-list validation on the category parameter, accepting only known-good category values.

- Apply the principle of least privilege to the database account used by the application (no unnecessary access to information_schema or file operations).
- Disable verbose error messages in production; return generic error responses to the client and log detailed errors server-side only.
- Deploy a Web Application Firewall (WAF) with SQL injection detection rules as a defense-in-depth measure.
- Incorporate automated static/dynamic application security testing (SAST/DAST) into the CI/CD pipeline to catch injection flaws before release.

6. Conclusion

This engagement successfully identified and exploited a Critical SQL Injection vulnerability in the category filtering functionality of the target application. Using a methodical UNION-based approach — determining column count, identifying valid comment syntax, and extracting data via the @@version system variable — the underlying MySQL database version was disclosed without authentication. The exercise demonstrates both the systematic methodology required to identify blind structural details of a vulnerable query and the significant risk that unsanitized, string-concatenated SQL queries pose in production environments. It is recommended that the remediation steps outlined in Section 5.6 be prioritized to eliminate this class of vulnerability.

— *End of Report* —